



**Gisma
University
of Applied
Sciences**

Assessment Submission Form

Student Number (If this is group work, please include the student numbers of all group participants)	LORENA BELLO – GH1041653
Assessment Title	INDIVIDUAL PROJECT
Module Code	B105B
Module Title	APPLIED STATISTICAL MODELING
Module Tutor	WILLIAM MORRISON
Date Submitted	16.09.2026

Declaration of Authorship

I declare that all material in this assessment is my own work except where there is clear acknowledgement and appropriate reference to the work of others.

I fully understand that the unacknowledged inclusion of another person's writings or ideas or works in this work may be considered plagiarism and that, should a formal investigation process confirm the allegation, I would be subject to the penalties associated with plagiarism, as per GISMA Business School, University of Applied Sciences' regulations for academic misconduct.

Signed. LORENA BELLO... Date16.09.2026.....

THE TOP 1000 IMDB MOVIES

1. DISCUSSION OF UNDERLYING BUSINESS PROBLEM:

Entertainment companies need to make high-stake decisions regarding what films to produce, license, promote or acquire, these decisions are made under the uncertainty of the public's reception and the commercial performance of these films. The production and acquisition of movies include large capitals with high failure rates. Doing a data-driven insight into what makes a film successful critically and commercially can reduce risks and sharpens the strategies that these companies will use for future projects.

For this business problem we can ask these questions:

- What makes a film critically and commercially successful? Is there a relationship between the ratings on websites like IMDB or Metascore, and Gross revenue?
- Which are the genres, actors or directors that constantly produce top-tier films?
- Is there a connection between runtime and certificates (age ratings) to the audience's reception?
- Is there a continuous trend over-time that explain what films reach an elite status?

2. SET UP OF HYPOTHESIS TO BE TESTED

- HYPOTHESIS 1: Critical rating vs. commercial issues
 - **H0**: There is **no** significant correlation between the IMDB_Rating and Gross revenue
 - **H1**: There is a significant correlation between the IMDB_Rating and Gross revenue
- HYPOTHESIS 2: Meta score vs. commercial success
 - **H0**: There is **no** significant correlation between the Meta_Score and Gross revenue
 - **H1**: There is a significant correlation between the Meta_Score and Gross revenue
- HYPOTHESIS 3: Genre and rating
 - **H0**: The mean of the IMDB_Rating does **not** differ significantly across genres
 - **H1**: The mean of the IMDB_Rating does differ significantly across genres
- HYPOTHESIS 4: Runtime and rating

- **H0:** There is **no** significant relationship between Runtime and IMDB_Rating
- **H1:** There is a significant relationship between Runtime and IMDB_Rating
- **HYPOTHESIS 5:** Certificates and gross revenues
 - **H0:** Gross revenue does **not** differ significantly across age ratings categories
 - **H1:** Gross revenue does differ significantly across age ratings categories
- **HYPOTHESIS 6:** Votes and gross revenue
 - **H0:** There is **no** significant correlation between No_of_Votes and Gross revenues
 - **H1:** There is a positive correlation between No_of_Votes and Gross revenues
- **HYPOTHESIS 7:** Trends over time
 - **H0:** The mean of IMDB_Rating of top-1000 films has **not** changed significantly across release decades
 - **H1:** The mean of IMDB_Rating of top-1000 films has changed significantly across release decades

3. DATA PRE-PROCESSING, SAMPLING AND CLEANING STEPS

- One of the first steps of cleaning will be making sure that every column has its correct data type.
 - Poster_Link = character
 - Series_Title = character
 - Released_Year = character
 - Certificate = character
 - Runtime = character
 - Genre = character
 - IMDB_Rating = numeric
 - Overview = character
 - Meta_Score = integer
 - Director = character
 - Star1 = character
 - Star2 = character
 - Star3 = character
 - Star4 = character
 - No_of_Votes = integer
 - Gross = character

```
> str(imdb_movies)
'data.frame': 1000 obs. of 16 variables:
 $ Poster_Link : chr "https://m.media-amazon.com/images/M/MV5BMDkYTC0MGETZmNmMC00ZDIzLWFmNTetODM1ZmRlYWwMwMwFmXkEyXkFqcGdeQXVyMTMxODk" | __truncated__ "https://m.media-amazon.com/images/M/MV5BM2MyNjYxNmUtYTAwNi00MTYxLWJmNWYtYzZlODY3ZTk3OTF1XkEyXkFqcGdeQXVyNzkwMjQ" | __truncated__ "https://m.media-amazon.com/images/M/MV5BMTMxNTMwODM0NF5BM15BanBnXkFtZTcwODAyMTk2Mw@@._V1_UX67_CR0,0,67,98_AL_.jpg" "https://m.media-amazon.com/images/M/MV5BMTMwMGQzZTItY2JlNC00OWZiLWIyMDctNDk2ZDQ2YjRjMwQ0XkEyXkFqcGdeQXVyNzkwMjQ" | __truncated__ ...
 $ Series_Title : chr "The Shawshank Redemption" "The Godfather" "The Dark Knight" "The Godfather: Part II"
 ...
 $ Released_Year: chr "1994" "1972" "2008" "1974" ...
 $ Certificate : chr "A" "A" "UA" "A" ...
 $ Runtime : chr "142 min" "175 min" "152 min" "202 min" ...
 $ Genre : chr "Drama" "Crime, Drama" "Action, Crime, Drama" "Crime, Drama" ...
 $ IMDB_Rating : num 9.3 9.2 9 9 9 8.9 8.9 8.9 8.8 8.8 ...
 $ Overview : chr "Two imprisoned men bond over a number of years, finding solace and eventual redemption through acts of common decency." "An organized crime dynasty's aging patriarch transfers control of his clandestine empire to his reluctant son." "When the menace known as the Joker wreaks havoc and chaos on the people of Gotham, Batman must accept one of the most demanding choices of his life: to save the city or to let it be consumed by Gotham's underworld." "The early life and career of Vito Corleone in 1920s New York City is portrayed, while his son, Michael, expands and solidifies his empire through a series of ruthless moves." | __truncated__ ...
 $ Meta_score : int 80 100 84 90 96 94 94 94 74 66 ...
 $ Director : chr "Frank Darabont" "Francis Ford Coppola" "Christopher Nolan" "Francis Ford Coppola" ...
 $ Star1 : chr "Tim Robbins" "Marlon Brando" "Christian Bale" "Al Pacino" ...
 $ Star2 : chr "Morgan Freeman" "Al Pacino" "Heath Ledger" "Robert De Niro" ...
 $ Star3 : chr "Bob Gunton" "James Caan" "Aaron Eckhart" "Robert Duvall" ...
 $ Star4 : chr "William Sadler" "Diane Keaton" "Michael Caine" "Diane Keaton" ...
 $ No_of_Votes : int 2343110 1620367 2303232 1129952 689845 1642758 1826188 1213505 2067042 1854740 ...
 $ Gross : chr "28,341,469" "134,966,411" "534,858,444" "57,300,000" ...
```

```
> sapply(imdb_movies, class)
Poster_Link      Series_Title Released_Year Certificate      Runtime      Genre      IMDB_Rating
"character"      "character"      "character"      "character"      "character"      "character"      "numeric"
Overview      Meta_score      Director      Star1      Star2      Star3      Star4
"character"      "integer"      "character"      "character"      "character"      "character"      "character"
No_of_Votes      Gross
"integer"      "character"
```

The columns `Released_Year`, `Runtime` and `Gross` should be numeric since they contain numbers and it makes analysis easier for both R and myself. But before transforming them into integer we have to check if there are any errors in said columns.

Firstly, with the help of the “tidyverse” and “stringr” packages, we find that in the film “Apollo 13” its `Released_Year` column doesn’t have any numerical values like the rest, due to having a string R transform the entire column into a string type. We replace that string with the release year so that the `Release_Year` column can be turned into an integer.

```
> bad_year_rows <- imdb_movies |>
+ subset(!str_detect(Released_Year, "[0-9]{4}$")) |>
+ subset(select = c(Series_Title, Released_Year, Certificate))
> bad_year_rows
  Series_Title Released_Year Certificate
1967 Apollo 13             PG          U

imdb_movies$Released_Year[imdb_movies$Series_Title == "Apollo 13" &
  imdb_movies$Released_Year == "PG"] <- "1995"
```

Checking the `Runtime` column, we find that the column is made up of number (number of minutes) and a string (the string “min”), that’s why the column is identified as a

character by R. To be able to transform this column into an integer we need to remove the “min”.

```
> head(imdb_movies$Runtime)
[1] "142 min" "175 min" "152 min" "202 min" "96 min"  "201 min"
```

Lastly, the Gross column presents a similar issue to Runtime because of the commas in between R reads it as a string, to transform the column into a numeric column we need to remove these commas. Gross is being changed to numeric rather than an integer because of how big these numbers are.

```
> head(imdb_movies$Gross)
[1] "28,341,469" "134,966,411" "534,858,444" "57,300,000" "4,360,000" "377,845,905"

3 imdb_movies$Released_Year <- as.integer(imdb_movies$Released_Year)
4 imdb_movies$Runtime <- as.integer(str_remove(imdb_movies$Runtime, " min"))
5 imdb_movies$Gross <- as.numeric(str_remove_all(imdb_movies$Gross, ","))
```

```
apply(imdb_movies, class)
Poster_Link Series_Title Released_Year Certificate Runtime Genre IMDB_Rating
"character" "character" "integer" "character" "integer" "character" "numeric"
Overview Meta_score Director Star1 Star2 Star3 Star4
"character" "integer" "character" "character" "character" "character" "character"
No_of_Votes Gross
"integer" "numeric"
```

After the transformation of datatype of those columns, we look for missing values in all columns.

```
> colSums(is.na(imdb_movies) | imdb_movies == "")
Poster_Link 0 Series_Title 0 Released_Year 0 Certificate 101 Runtime 0 Genre 0 IMDB_Rating 0
Overview 0 Meta_score 157 Director 0 Star1 0 Star2 0 Star3 0 Star4 0
No_of_Votes 0 Gross 169
```

We can see that 3 columns “Certificate”, “Meta_Score” and “Gross” have missing values. Now that we know that we have missing values, we need to handle these values. Dropping these values is one strategy for dealing with them but I don’t believe it to be the best one due to the amount of missing values, dropping them would make the dataset lose a great amount of information that could jeopardize the analysis.

There is also the risk of bias, the missing values of these columns are not all random some of the information missing could be attributed to those records not being available at the time. For example, the Gross revenue of older films might’ve not been recorded as it is recorded on the current era of movies.

For “Certificate” I believe the best strategy would be making these missing values its own category that being “Not Rated” as a way to describe that those films don’t a certificate on record.

```
imdb_movies$Certificate[imdb_movies$Certificate == ""] <- "Not Rated"
```

In the “Meta_Score” and “Gross” columns, which have the highest missing values the best strategy in my opinion is when testing the hypothesis that specifically rely on those variables, we omit those missing values so they don’t affect the hypothesis.

When checking the “Certificate” column we can see that there are 17 distinct values but they are from different inconsistent ratings systems this including the new category created from missing values “Not Rated”. Using the “Certificate” column like this for hypothesis (H5) testing could put at risk the results we would get from it, so the best way to deal with them is to create 3 broader categories that can accommodate similar certificates in one, the categories being “General Audience”, “Mature” and “Not Rated”. This change allows each group to have a sufficient sample size for testing and it also fits the underlying business problem since it’s not limited by regional values.

```
> table(imdb_movies$Certificate)
```

16	A	Approved	G	GP	Not Rated	Passed	PG	PG-13	R
1	197	11	12	2	101	34	37	43	146
TV-14	TV-MA	TV-PG	U	U/A	UA	Unrated			
1	1	3	234	1	175	1			

```
imdb_movies <- imdb_movies %>%
  mutate(Certificate_Group = case_when(
    Certificate %in% c("U", "G", "UA", "U/A", "PG", "TV-PG", "Approved", "Passed", "GP") ~ "General Audience",
    Certificate %in% c("A", "R", "PG-13", "TV-14", "TV-MA", "16", "Unrated") ~ "Mature",
    TRUE ~ "Other"
  ))
```

```
> table(imdb_movies$Certificate_Group)
```

General Audience	Mature	Not Rated
509	390	101

Another problem that can surface when testing the hypothesis (H3) for the “Genre” column is that there are multiple genres separated by commas rather than one genre per row. To simplify this for group comparison, we extract the first genre listed as new “Primary_Genre” column this gives each film a representative category.

```
> head(imdb_movies$Genre)
```

[1] "Drama"	"Crime, Drama"	"Action, Crime, Drama"
[4] "Crime, Drama"	"Crime, Drama"	"Action, Adventure, Drama"

```
imdb_movies$Primary_Genre <- str_trim(str_split_fixed(imdb_movies$Genre, ",", 2)[, 1])
```

Following the same pattern, to make hypothesis (H7) testing easier, we derive a decade variable for the “Released_Year” column. This will give each film a column specifying the decade the film was released.

```
imdb_movies$Decade <- floor(imdb_movies$Released_Year / 10) * 10
```

After doing these steps, we can start removing and flagging the duplicates on the dataset.

```
> sum(duplicated(imdb_movies$Series_Title))
[1] 1
> imdb_movies <- imdb_movies %>% distinct(Series_Title, Released_Year, .keep_all = TRUE)
sum(duplicated(imdb_movies$Series_Title))
imdb_movies <- imdb_movies %>% distinct(Series_Title, Released_Year, .keep_all = TRUE)
```

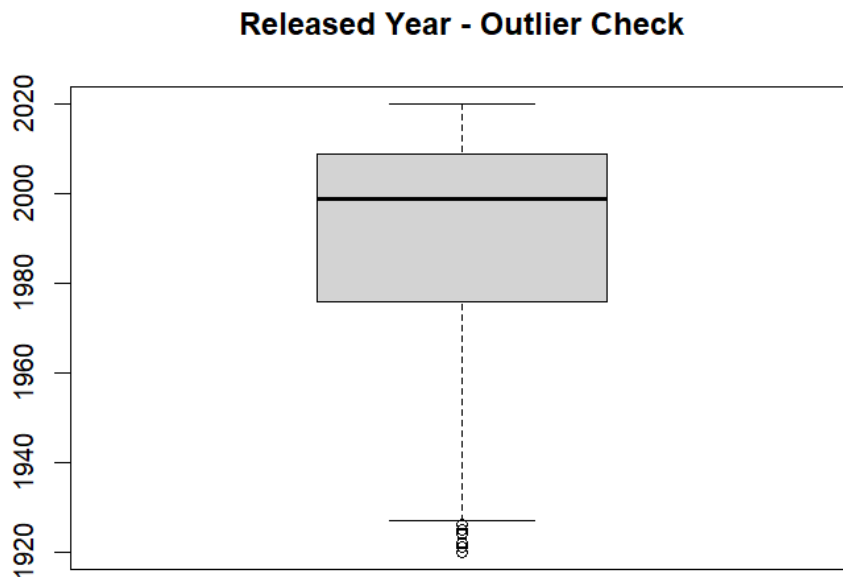
Next step should be dropping columns that won't contribute to our analysis like "Poster_Link" and "Overview"

```
imdb_clean <- imdb_movies %>% select(-Poster_Link, -Overview)
```

Now we check the outliers of all our numeric columns.

- Released_Year

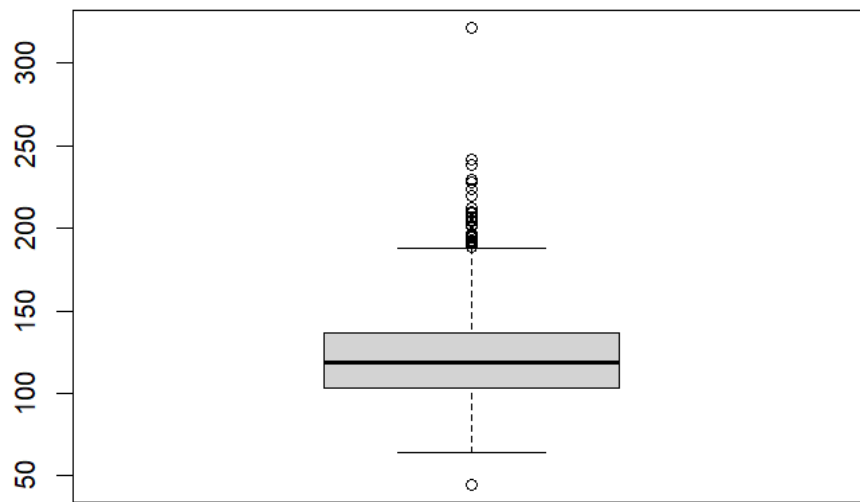
```
boxplot(imdb_clean$Released_Year, main = "Released Year - Outlier Check")
```



- Runtime

```
boxplot(imdb_clean$Runtime, main = "Runtime - Outlier Check")
```

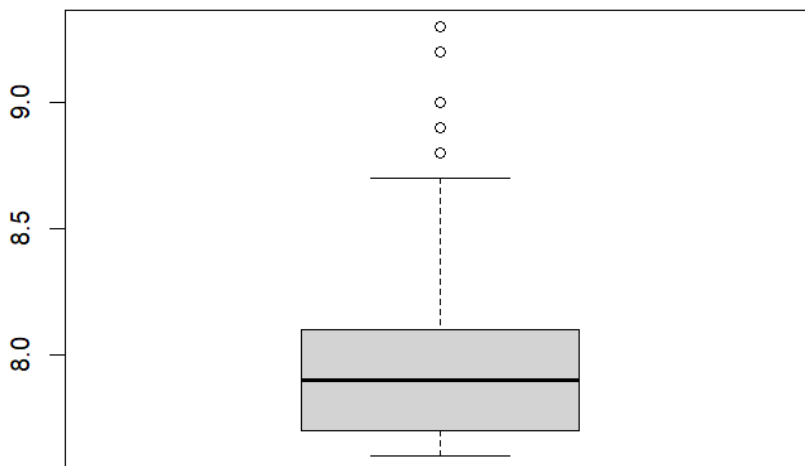
Runtime - Outlier Check



- IMDB_Rating

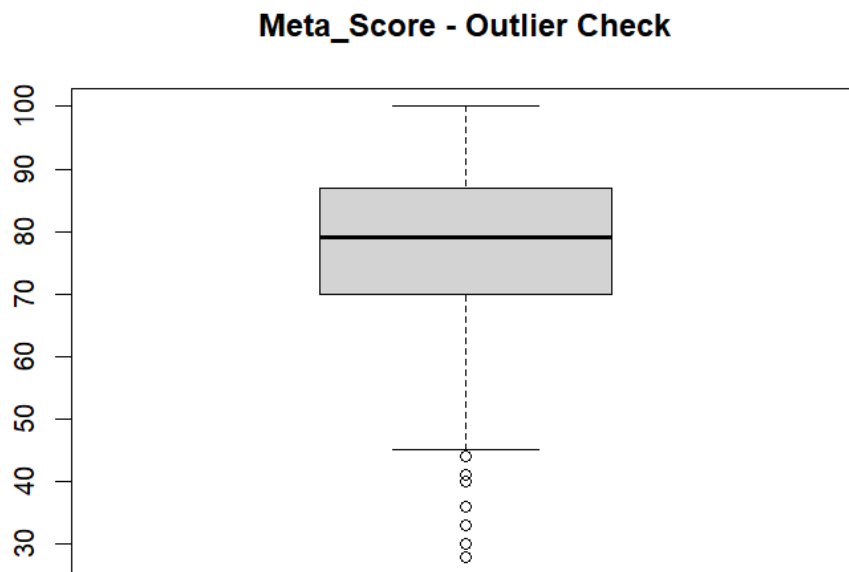
```
boxplot(imdb_clean$IMDB_Rating, main = "IMDB_Rating - Outlier Check")
```

IMDB_Rating - Outlier Check



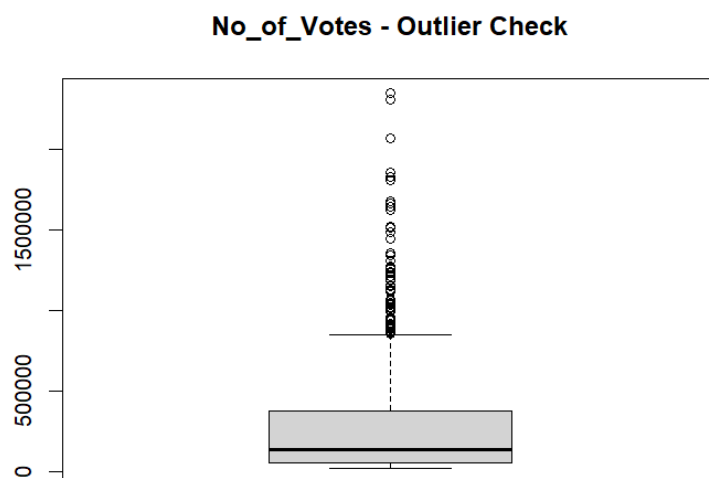
- Meta_Score

```
boxplot(imdb_clean$Meta_score, main = "Meta_Score - Outlier Check")
```



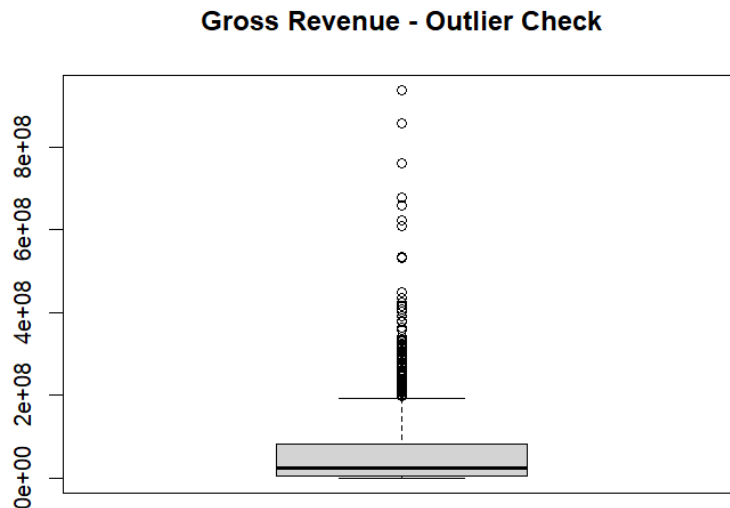
- No_of_Votes

```
boxplot(imdb_clean$No_of_Votes, main = "No_of_Votes - Outlier Check")
```



- Gross Revenue (Given the business context really high values of blockbuster earning more than regular films, these values are seen as genuine data and not errors)

```
boxplot(imdb_clean$Gross, main = "Gross Revenue - Outlier Check")
```



In regards of sampling for this dataset, I don't consider that it's necessary due to only being 1000 rows, which is relatively small in comparison to most datasets. Adding to that, due to the hypothesis testing and data analysis task to be done, the complete dataset is needed. Lastly, the dataset itself is already curated sample, sampling it further would hurt the results.

4. EXPLORATORY DATA ANALYSIS, INCLUDING DESCRIPTIVE STATISTICS

- Firstly, we will see the summary of all columns. With the summary () function we can see the different measures of our descriptive statistics of all our variables but we will be focusing on more on the results of the numeric columns.

```
> summary(imdb_clean)
  Series_Title   Released_Year   Certificate   Runtime   Genre   IMDB_Rating
Length :1000   Min. :1920   Length :1000   Min. : 45.0   Length :1000   Min. :7.600
N.unique : 999   1st Qu.:1976   N.unique : 17   1st Qu.:103.0   N.unique : 202   1st Qu.:7.700
N.blank : 0     Median :1999   N.blank : 0     Median :119.0   N.blank : 0     Median :7.900
Min.nchar: 2     Mean :1991     Min.nchar: 1     Mean :122.9     Min.nchar: 5     Mean :7.949
Max.nchar: 68    3rd Qu.:2009   Max.nchar: 9     3rd Qu.:137.0   Max.nchar: 29    3rd Qu.:8.100
Max. :2020
Max. :321.0
Max. :9.300

  Meta_score   Director   Star1   Star2   Star3   Star4
Min. : 28.00   Length :1000   Length :1000   Length :1000   Length :1000   Length :1000
1st Qu.: 70.00   N.unique : 548   N.unique : 660   N.unique : 841   N.unique : 891   N.unique : 939
Median : 79.00   N.blank : 0     N.blank : 0     N.blank : 0     N.blank : 0     N.blank : 0
Mean : 77.97    Min.nchar: 7     Min.nchar: 4     Min.nchar: 4     Min.nchar: 4     Min.nchar: 4
3rd Qu.: 87.00   Max.nchar: 32    Max.nchar: 25    Max.nchar: 25    Max.nchar: 27    Max.nchar: 27
Max. :100.00
NAS :157

  No_of_Votes   Gross   Certificate_Group   Primary_Genre   Decade
Min. : 25088   Min. : 1305   Length :1000   Length :1000   Min. :1920
1st Qu.: 55526   1st Qu.: 3253559   N.unique : 3     N.unique : 14   1st Qu.:1970
Median : 138549   Median : 23530892   N.blank : 0     N.blank : 0     Median :1990
Mean : 273693    Mean : 68034751    Min.nchar: 6     Min.nchar: 5     Mean :1987
3rd Qu.: 374161   3rd Qu.: 80750894   Max.nchar: 16    Max.nchar: 9     3rd Qu.:2000
Max. :2343110    Max. :936662225    Max.nchar: 16    Max.nchar: 9     Max. :2020
NAS :169
```

- With the describe() function we will more detailed statistics of these columns.

```
> describe(imdb_clean %>% select(IMDB_Rating, Meta_score, Runtime,
+                               Gross, No_of_Votes, Released_Year))
  vars      n      mean      sd      median      trimmed      mad      min      max
IMDB_Rating 1 1000      7.95      0.28      7.9      7.92      0.30      7.6      9.3
Meta_score   2  843     77.97     12.38     79.0     78.68     11.86     28.0    100.0
Runtime      3 1000    122.89     28.09    119.0    120.28     25.20     45.0    321.0
Gross        4  831 68034750.87 109750043.19 23530892.0 43494539.43 33752523.19 1305.0 936662225.0
No_of_Votes  5 1000  273692.91  327372.70 138548.5  207176.02 146278.51 25088.0 2343110.0
Released_Year 6 1000   1991.22     23.29   1999.0    1994.03     20.76   1920.0   2020.0

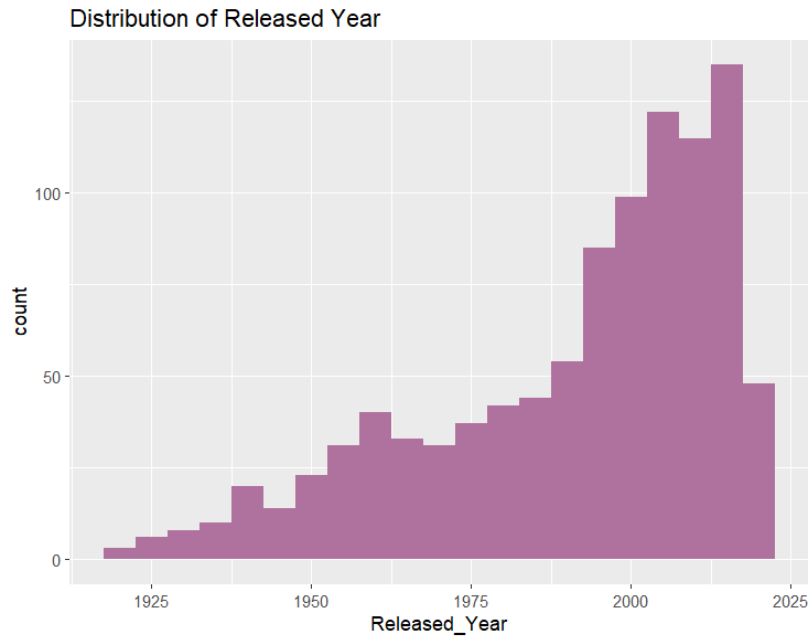
  range skew kurtosis      se
IMDB_Rating 1.7 1.01 1.41 0.01
Meta_score  72.0 -0.60 0.40 0.43
Runtime    276.0 1.20 3.39 0.89
Gross    936660920.0 3.12 13.78 3807186.82
No_of_Votes 2318022.0 2.29 6.83 10352.43
Released_Year 100 0 -0.94 -0.03 0.74
```

- Seeing the statistics for variables relevant to hypothesis is also useful

```
> mean(imdb_clean$IMDB_Rating, na.rm = TRUE)
[1] 7.9493
> sd(imdb_clean$IMDB_Rating, na.rm = TRUE)
[1] 0.2754912
> median(imdb_clean$Gross, na.rm = TRUE)
[1] 23530892
> sd(imdb_clean$Gross, na.rm = TRUE)
[1] 109750043
```

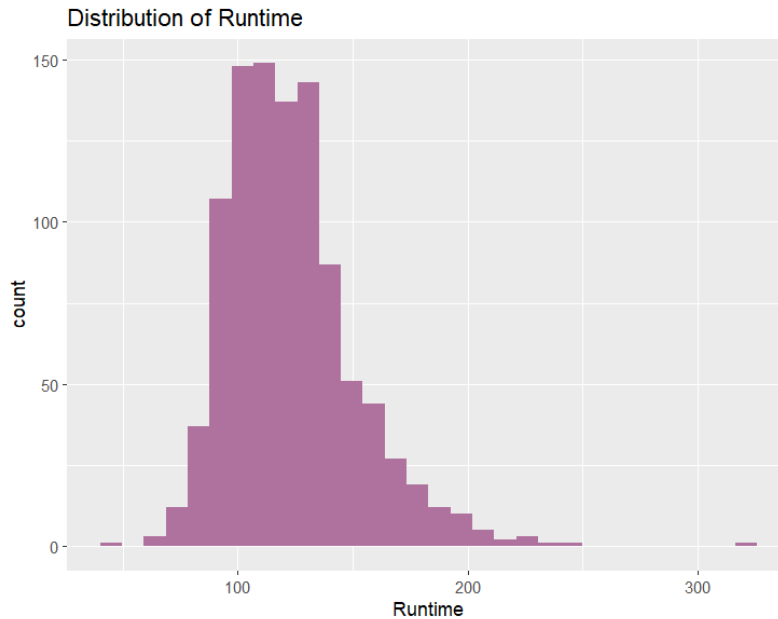
- For analyzing the summary results, we also check the histograms. For categorical values we use a bar chart
- For the Released Year column, we can see that it spans exactly a century from 1920 to 2020, with a mean of 1999 and a median of 1991. Meaning that it's more left skewed leaning more to recent decades, but despite this it includes many older films. This gives our H7 a wide time range to test across trends.

```
ggplot(imdb_clean, aes(x = Released_Year)) +
  geom_histogram(binwidth = 5, fill = "#AF719D") +
  labs(title = "Distribution of Released Year")
```



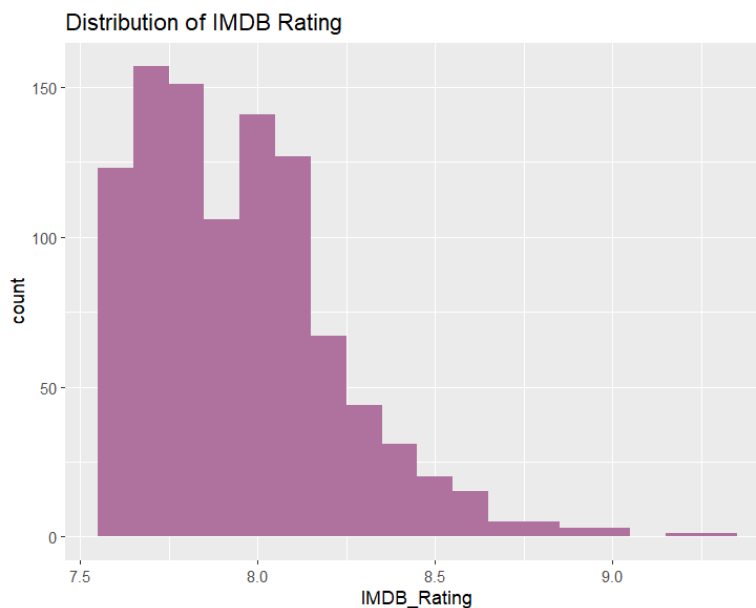
- For the Runtime column, it ranges from 45 to 321 min with a mean of 122.9 and a median of 119. The mean is barely higher than the median making it slightly right skewed as shown by the histogram which shows most films are in between the 100-200 minutes. There's only a small number of long films that increase the average. A median of 119 minutes aligns with a typical runtime expected for mainstream film length

```
ggplot(imdb_clean, aes(x = Runtime)) +  
  geom_histogram(bins= 30, fill = "#AF719D") +  
  labs(title = "Distribution of Runtime")
```



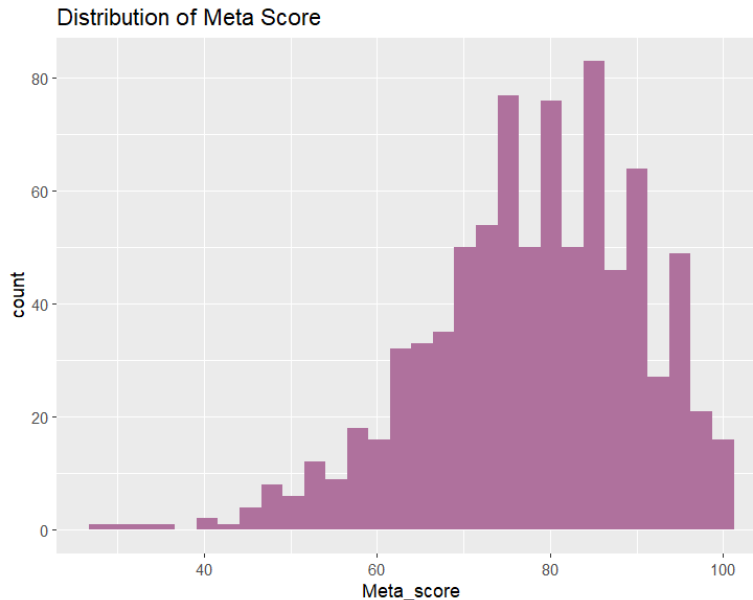
- For the IMDB_Rating column, a range from 7.6 to 9.3, a mean of 7.95 which is very close to the median of 7.9. The range being this close is expected due to the dataset being prefiltered with the top 1000 films instead of a random sample of films. The restricted range will play an important part in correlation tests that include this variable (H1, H4, H7).

```
ggplot(imdb_clean, aes(x = IMDB_Rating)) +  
  geom_histogram(binwidth = 0.1, fill = "#AF719D") +  
  labs(title = "Distribution of IMDB Rating")
```



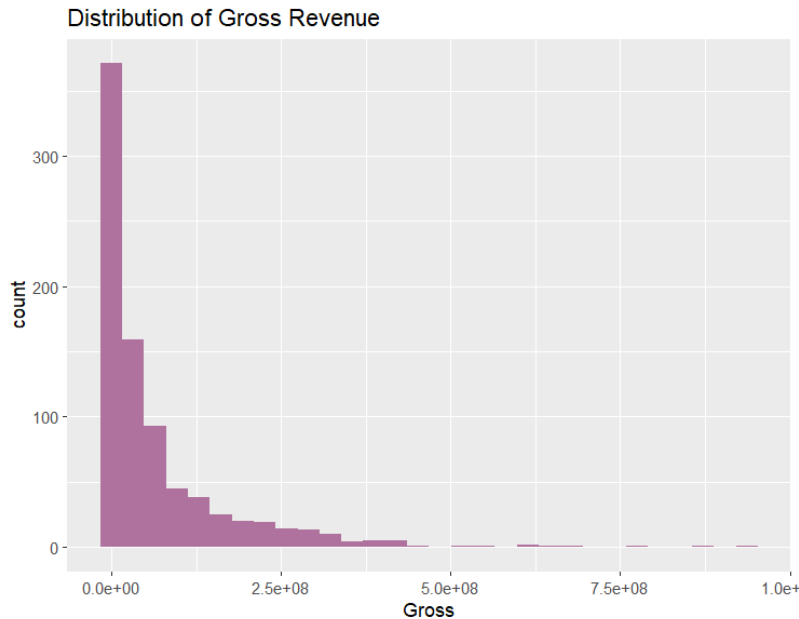
- For the Meta_Score column, the range in this column is wider going from 28 to 100 in comparison with the IMDB_Rating range. It has a mean of 77.97 which is just somewhat below the median of 79. We can observe a mild left-skew which suggests a minority of low critic scores. The wider spread in Meta_Score shows a contrast between audience opinion and critic opinion.

```
ggplot(imdb_clean, aes(x = Meta_score)) +  
  geom_histogram(bins= 30, fill = "#AF719D") +  
  labs(title = "Distribution of Meta Score")
```



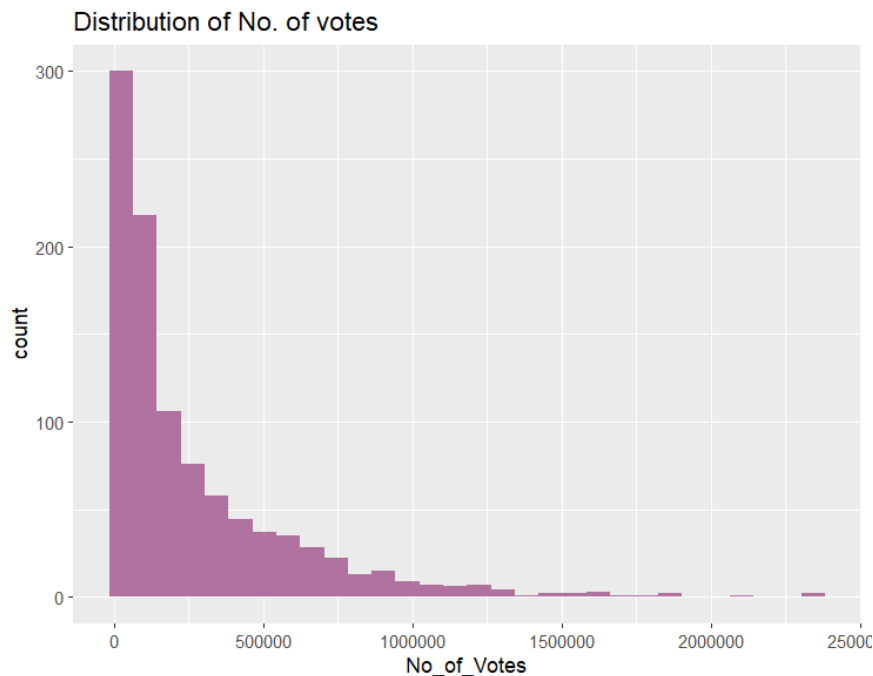
- For the Gross column, we can see that this column is strongly right-skewed with a mean of 68M and lower median of 23.5M. The max of 936.6M is incredibly higher than the 3rd quartile of 80.75M. This confirms an expected pattern that only a small number of blockbusters earn a lot more than the average top-rated films.

```
ggplot(imdb_clean, aes(x = Gross)) +  
  geom_histogram(bins= 30, fill = "#AF719D") +  
  labs(title = "Distribution of Gross Revenue")
```



- For the No_of_Votes column, this column is visibly right-skewed with a mean of 273,693 and a median of 138,549 due to the same reason as Gross, popular titles draw more engagement than other films on the list.

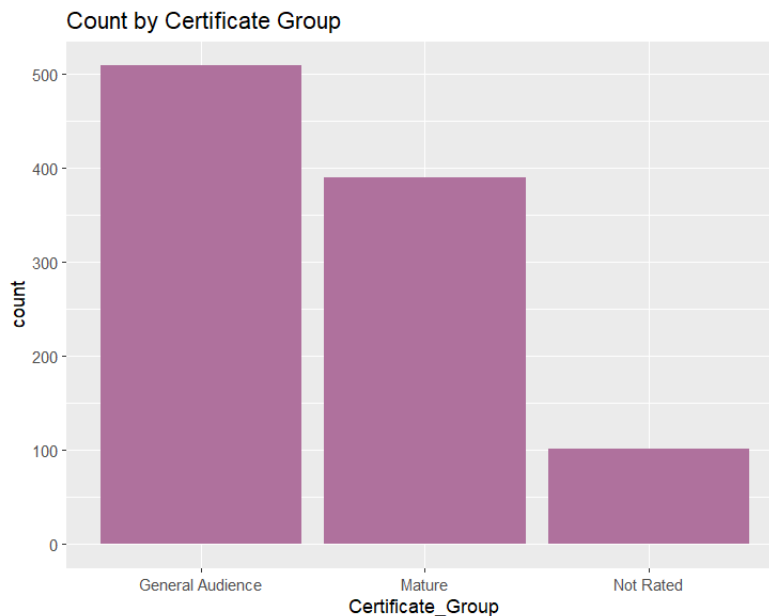
```
ggplot(imdb_clean, aes(x = No_of_Votes)) +  
  geom_histogram(bins= 30, fill = "#AF719D") +  
  labs(title = "Distribution of No. of votes")
```



- For the Certificate_Group variable, that was created by grouping 17 raw certificates into 3 categories. The bar chart shows “General Audience” as the most common category with 509 films, while the smallest category is “Not

Rated” which was expected because it consists of those films that originally had missing data (101 of 1000). This is relevant for when we test if the Gross revenue changes across certifications groups (H5). The small sample size in “Not Rated” could reduce the statistical power of the comparison, and this result should be interpreted with caution.

```
ggplot(imdb_clean, aes(x = Certificate_Group)) +  
  geom_bar(fill = "#AF719D") +  
  labs(title = "Count by Certificate Group")
```

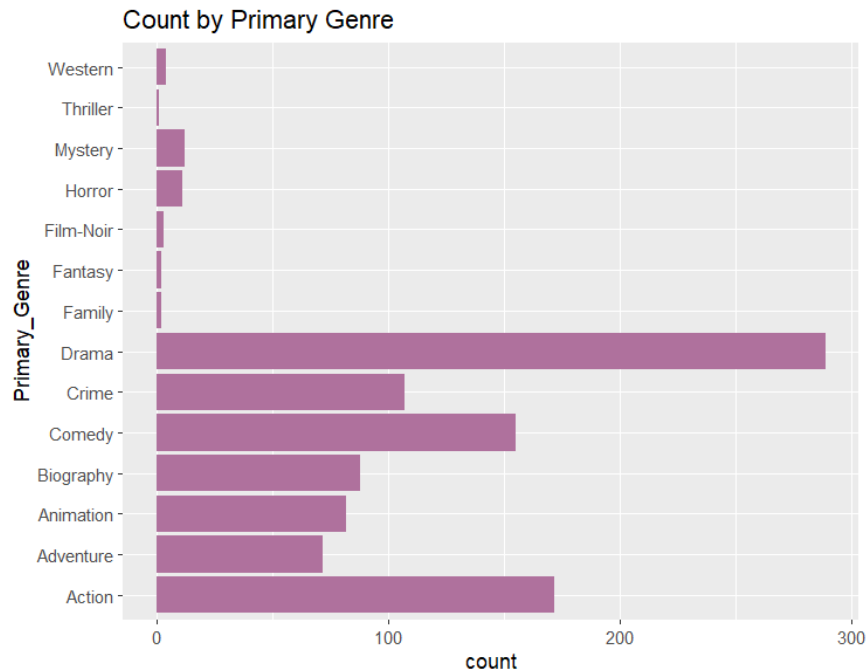


```
> table(imdb_clean$Certificate_Group)
```

General Audience	Mature	Not Rated
509	390	101

- For the Primary_Genre variable, which consists of the first genre listed for each film, in this bar chart we can notice a clear imbalance across the 14 categories. “Drama” is the most common genre by far with more than 200 films, followed by “Action” with 172 films and “Comedy” with 155 films. On the other hand, genres like “Thriller”, “Fantasy”, “Family”, “Film-Noir”, “Western”, “Horror” and “Mystery” have a smaller number of films, with some being barely visible on the chart. This imbalance will play a major part in H3, which compares the mean of IMDB_Rating across genres. Well represented genres will produce statically reliable estimations, while those with few films will have less reliable estimates and bigger uncertainty, like the previous variable any difference that includes this group have to interpreted with caution.

```
ggplot(imdb_clean, aes(x = Primary_Genre)) +  
  geom_bar(fill = "#AF719D") +  
  coord_flip() +  
  labs(title = "Count by Primary Genre")
```

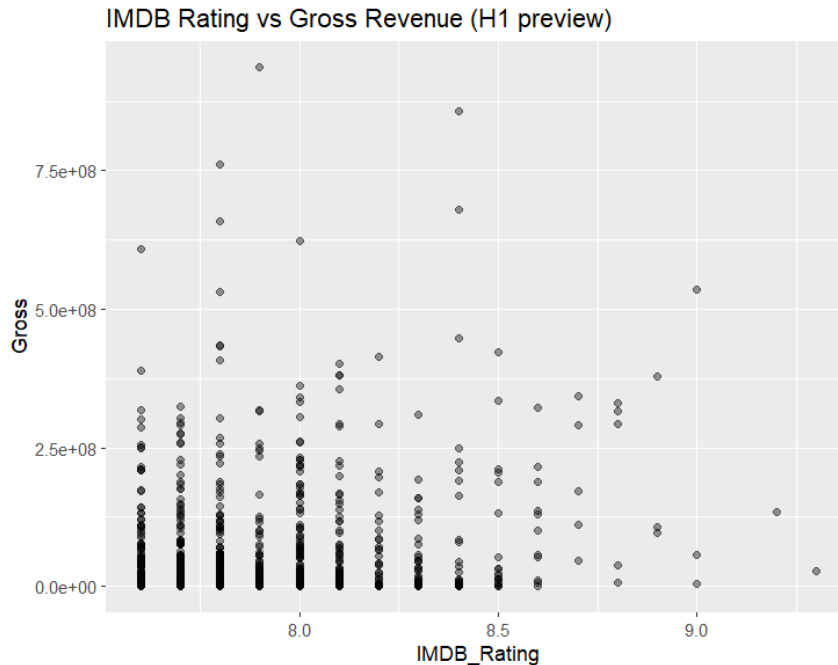


```
> table(imdb_clean$Primary_Genre)
```

Action	Adventure	Animation	Biography	Comedy	Crime	Drama	Family	Fantasy	Film-Noir
172	72	82	88	155	107	289	2	2	3
Horror	Mystery	Thriller	Western						
11	12	1	4						

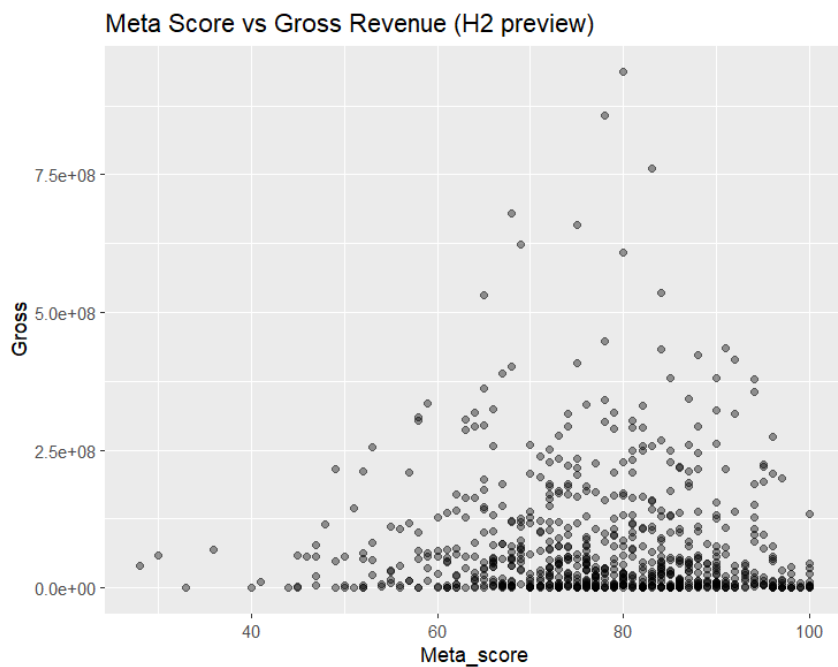
- After analyzing these columns with their histograms and bar charts, we see if there is a relationship between specific variables before testing it.
- IMDB Rating vs. Gross Revenue (H1): There is not clear upward or downward trend from the scattered points. High earning films aren't concentrated at the high end but they are rather across the whole rating range. This suggests a weak relationship between rating and revenue.

```
ggplot(imdb_clean, aes(x = IMDB_Rating, y = Gross)) +
  geom_point(alpha = 0.4) +
  labs(title = "IMDB Rating vs Gross Revenue (H1 preview)")
```



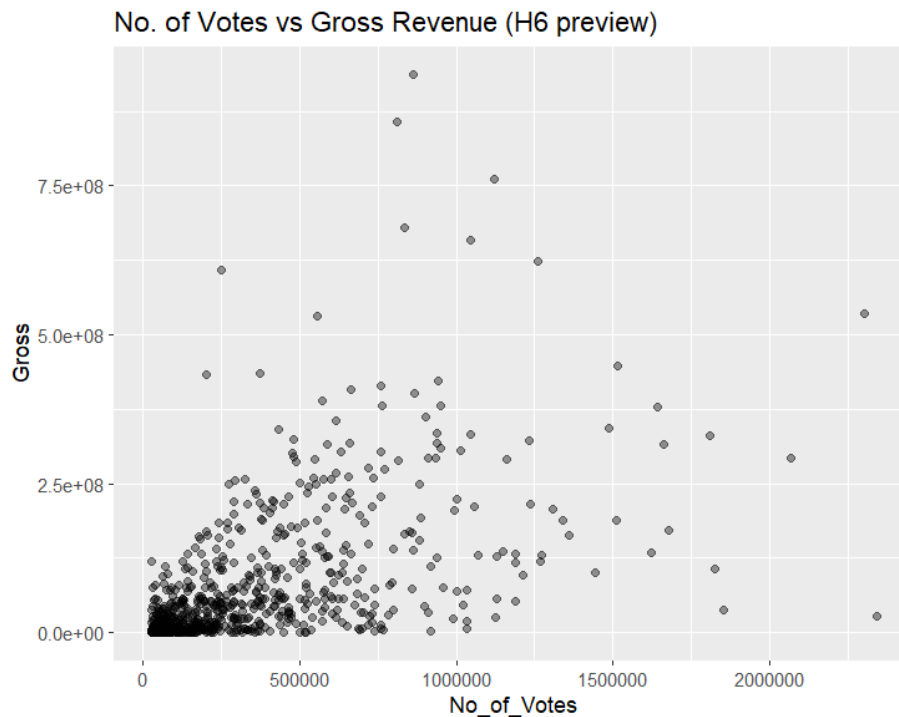
- Meta Score vs. Gross Revenue (H2): In comparison with the previous scatterplot this shows a sort of triangular shape, at low Meta_Scores, Gross stays consistently low but when Meta_Scores increases the range of the Gross values increases as well. This suggests a weak positive relationship, but most importantly the shape shows a non-constant variance.

```
ggplot(imdb_clean, aes(x = Meta_score, y = Gross)) +  
  geom_point(alpha = 0.4) +  
  labs(title = "Meta Score vs Gross Revenue (H2 preview)")
```



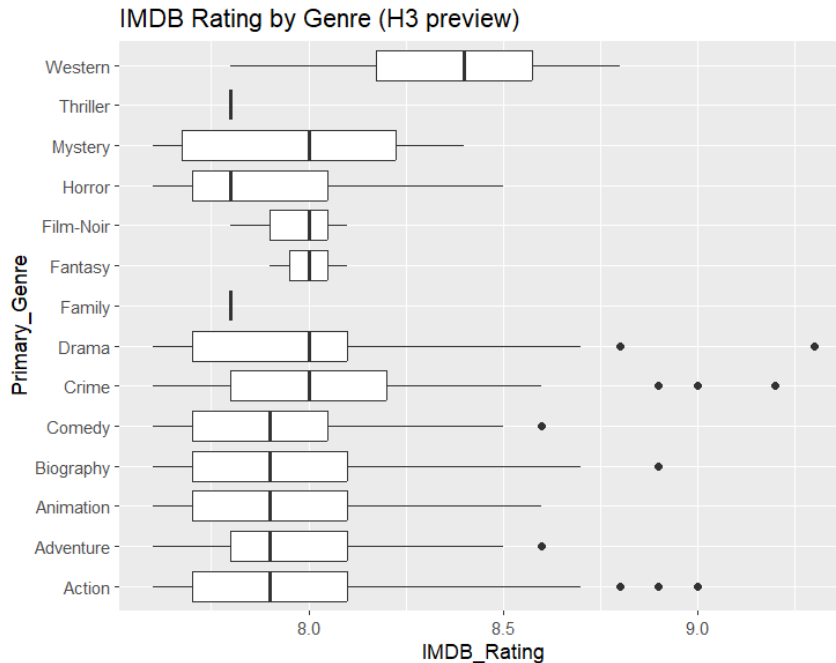
- No. of Votes vs. Gross Revenue (H6): The scatterplot shows a clear upward trend, when votes increase we can see the Gross visibly trend higher despite there still being a lot of scatter. This suggests a strong positive relationship, demonstrating that the more votes/engagements the more box office revenue.

```
ggplot(imdb_clean, aes(x = No_of_Votes, y = Gross)) +  
  geom_point(alpha = 0.4) +  
  labs(title = "No. of Votes vs Gross Revenue (H6 preview)")
```



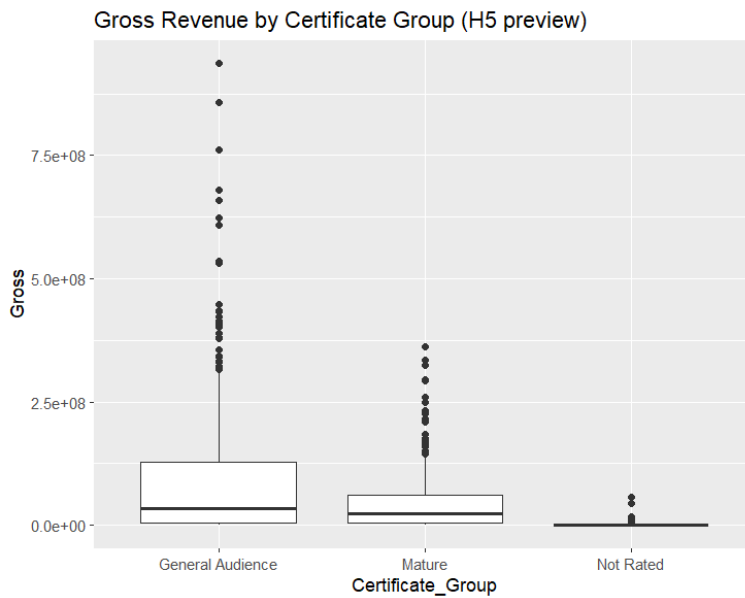
- Now we compare variables to see the statistical differences
- IMDB Rating by Genre (H3): The boxplot shows most genres gathered around a similar median, with the “Western” genre being visibly higher. Still, several genres like “Thriller” and “Family” are represented by a single film so they won't pose much of a statistical significance. This suggests H3 could find some genre-level differences.

```
ggplot(imdb_clean, aes(x = Primary_Genre, y = IMDB_Rating)) +  
  geom_boxplot() +  
  coord_flip() +  
  labs(title = "IMDB Rating by Genre (H3 preview)")
```



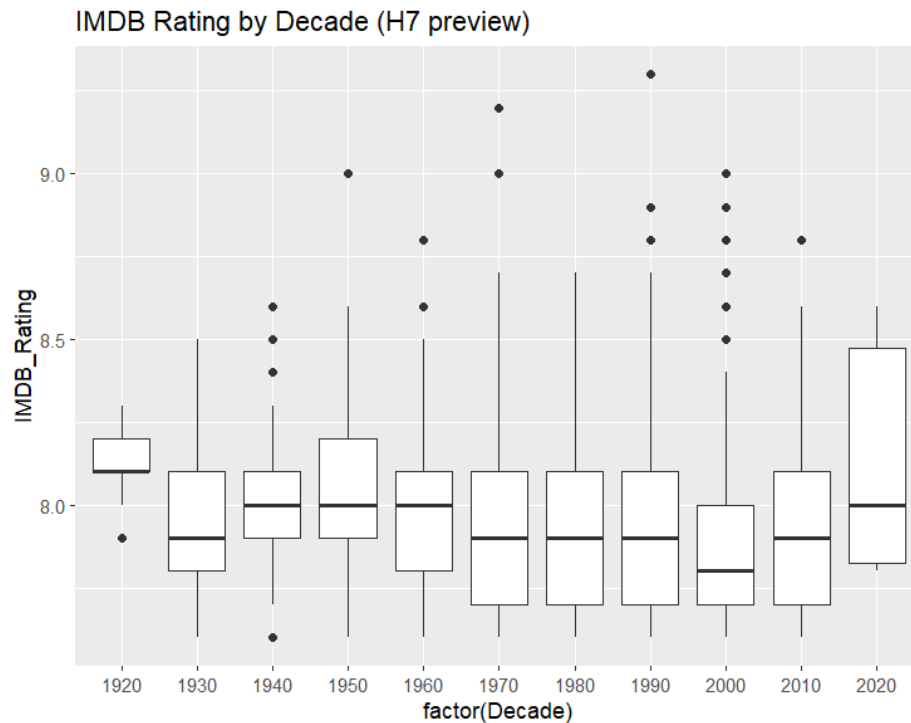
- Gross by Certificate Group (H5): “General Audience” has the highest median Gross in this boxplot and it also possesses the widest range of high-grossing outliers, while “Mature” films show a lower spread. Siting at the lowest is “Not Rated” this is consistent with the fact that this variable consists of films with missing certification data. Visually this suggests that Certificate_Group is relevant to Gross Revenue

```
ggplot(imdb_clean, aes(x = Certificate_Group, y = Gross)) +
  geom_boxplot() +
  labs(title = "Gross Revenue by Certificate Group (H5 preview)")
```

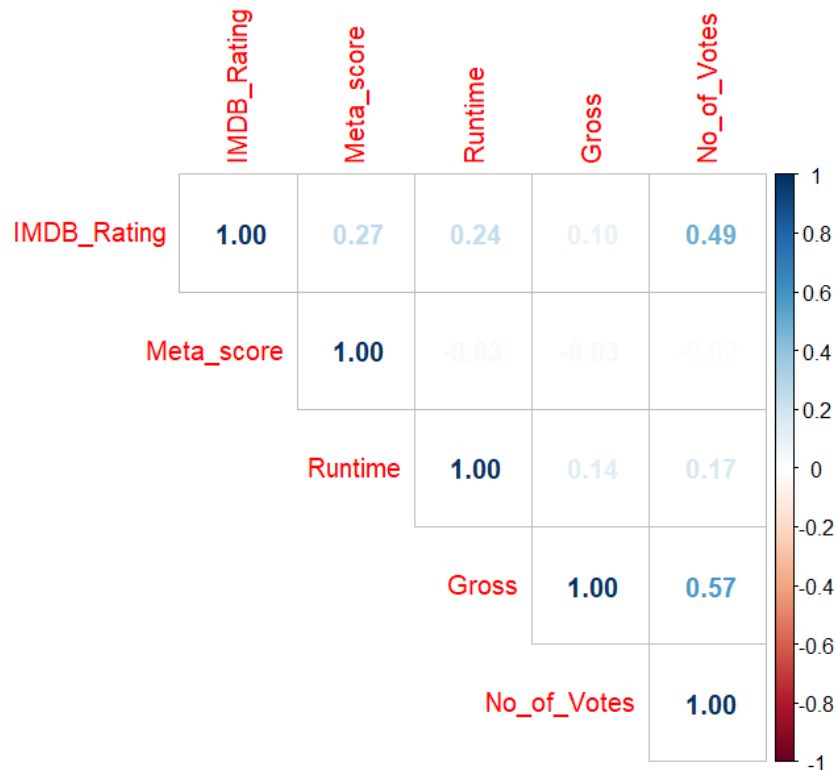


- IMDB_Rating by Decade (H7): There is a relatively stable IMDB_Rating median across most decades, with only the earliest and recent decades appearing modestly higher. This suggests little evidence of a strong overall trend in film rating overtime.

```
ggplot(imdb_clean, aes(x = factor(Decade), y = IMDB_Rating)) +  
  geom_boxplot() +  
  labs(title = "IMDB Rating by Decade (H7 preview)")
```



- Lastly, we see how the numeric values are correlated with one another
`corrplot(num_vars, method = "number", type = "upper")`



With this correlation matrix we are able to confirmed what the previous scatterplots suggested. The weakest correlations are IMDB_Rating and Gross with a correlation of 0.10 and, Meta_Score and Gross with a correlation of 0.03, this suggests that neither H1 nor H2 are likely to reveal a strong relationship in the tests. In comparison, No_of_Votes and Gross have the highest correlation of 0.57, reinforcing the strong relationship we saw in the upward trend in the scatterplot.

Other correlation values worth mentioning are those of the Runtime variable which are all weak consisting of 0.24, 0.14, 0.17 and -0.03. No_of_Votes also has another strong correlation, but this time with IMDB_Rating and a correlation of 0.49

5. CHOOSING AN APPROPRIATE INFERENTIAL TEST WITH ASSUMPTION VALIDATION

- SIMPLE LINEAR REGRESSION: H1 (Gross and IMDB_Rating), H2 (Gross and Meta_Score), H4 (IMDB_Rating and Runtime) and H6 (Gross and No_of_Votes) all of these hypotheses involve a continuous dependent variable and a continuous independent variable, so each will be tested with a Simple Linear Regression with the significance evaluated through a t-test on the slope coefficient. The assumptions that will be validated for

each model are linearity, homoscedasticity, normality of residuals and independence. On the EDA, we were able to see the Gross's right skew and how it can pose a problem for the homoscedasticity and normality assumptions, this will be confirmed through the diagnostic of residuals and if it is the case, it will be addressed with a log transformation.

- ONE-WAY ANOVA: H3 (IMDB_Rating across Genre), H5 (Gross across Certificate_Group) and H7 (IMDB_Rating across Decade) each compares a continuous variable with a categorical value with more than two independent groups, so it will be tested with a One-Way ANOVA. The assumptions validated are normality of residuals, homogeneity of variance, and independence. Imbalance due to the group size across all three is a challenge, so any equal variance results for these hypotheses need to be interpreted with caution

6. APPLICATION OF STATISTICAL TEST

- H1 APPLICATION TEST:

```
> model_h1 <- lm(Gross ~ IMDB_Rating, data = imdb_clean)
> summary(model_h1)
```

Call:

```
lm(formula = Gross ~ IMDB_Rating, data = imdb_clean)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-102820436	-62517252	-41910193	17363997	870372054

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-227376280	106535840	-2.134	0.03311 *
IMDB_Rating	37172969	13397415	2.775	0.00565 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

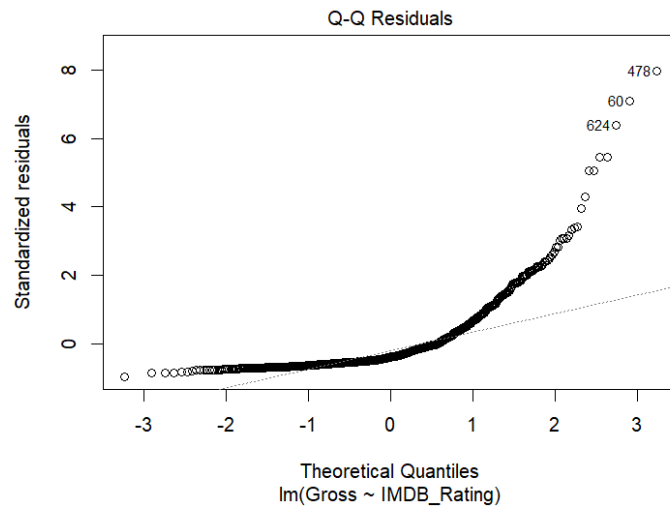
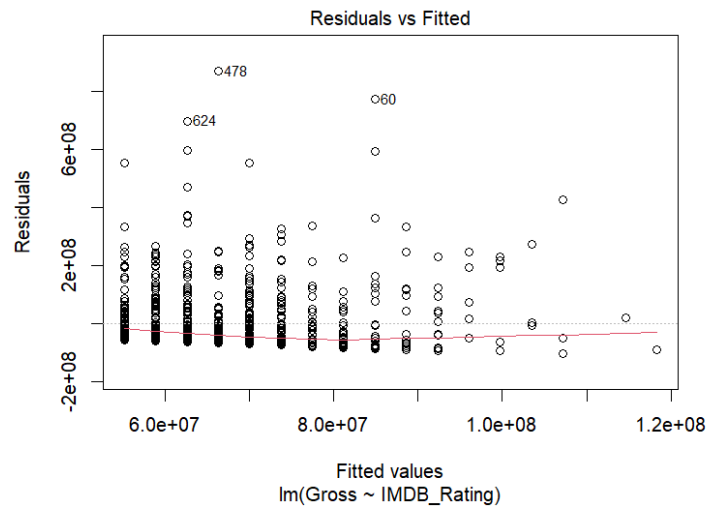
Residual standard error: 109300000 on 829 degrees of freedom
(169 observations deleted due to missingness)

Multiple R-squared: 0.009201, Adjusted R-squared: 0.008006

F-statistic: 7.699 on 1 and 829 DF, p-value: 0.005651

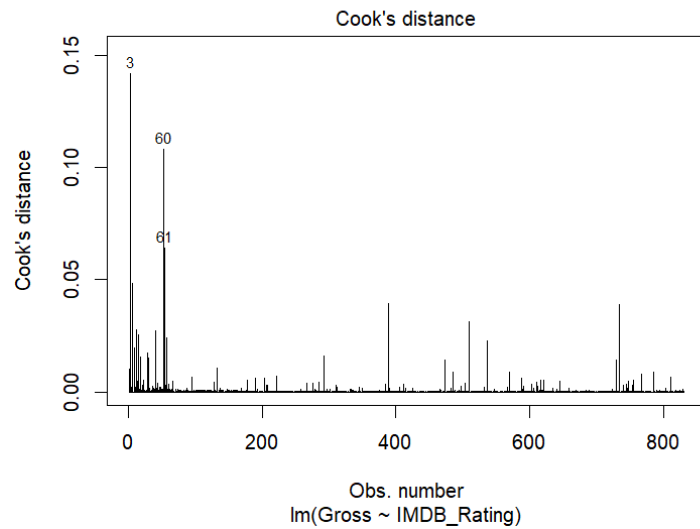
- ASSUMPTION CHECKS

```
plot(model_h1, which = 1)
plot(model_h1, which = 2)
shapiro.test(residuals(model_h1))
plot(model_h1, which = 4)
```



Shapiro-wilk normality test

```
data: residuals(model_h1)
W = 0.67248, p-value < 2.2e-16
```



LOG TRANSFORM

```
> summary(model_h1_log)
```

Call:

```
lm(formula = log(Gross) ~ IMDB_Rating, data = imdb_clean)
```

Residuals:

Min	1Q	Median	3Q	Max
-9.2541	-1.4279	0.5549	1.7819	4.2328

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	16.66501	2.35064	7.090	2.88e-12 ***
IMDB_Rating	-0.03038	0.29560	-0.103	0.918

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.412 on 829 degrees of freedom

(169 observations deleted due to missingness)

Multiple R-squared: 1.274e-05, Adjusted R-squared: -0.001194

F-statistic: 0.01056 on 1 and 829 DF, p-value: 0.9182

```
> shapiro.test(residuals(model_h1_log))
```

Shapiro-Wilk normality test

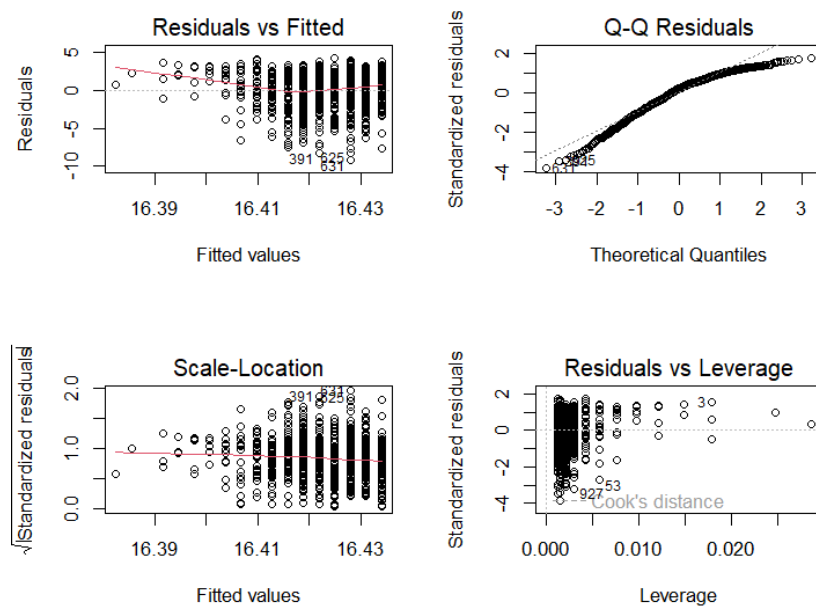
data: residuals(model_h1_log)

W = 0.94431, p-value < 2.2e-16

```
par(mfrow = c(2, 2))
```

```
plot(model_h1_log)
```

```
par(mfrow = c(1, 1))
```



• H2 APPLICATION TEST:

```
> model_h2 <- lm(Gross ~ Meta_score, data = imdb_clean)
> summary(model_h2)
```

Call:
lm(formula = Gross ~ Meta_score, data = imdb_clean)

Residuals:

	Min	1Q	Median	3Q	Max
	-87279147	-68401159	-43159626	23025583	862414862

Coefficients:

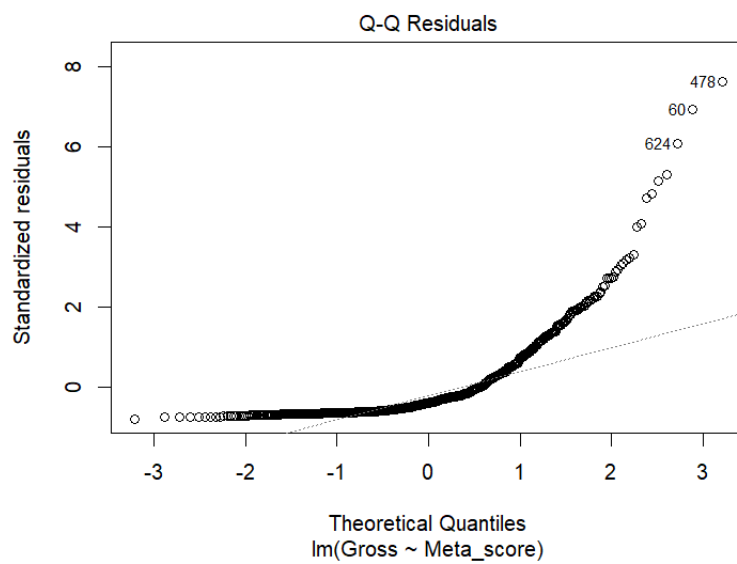
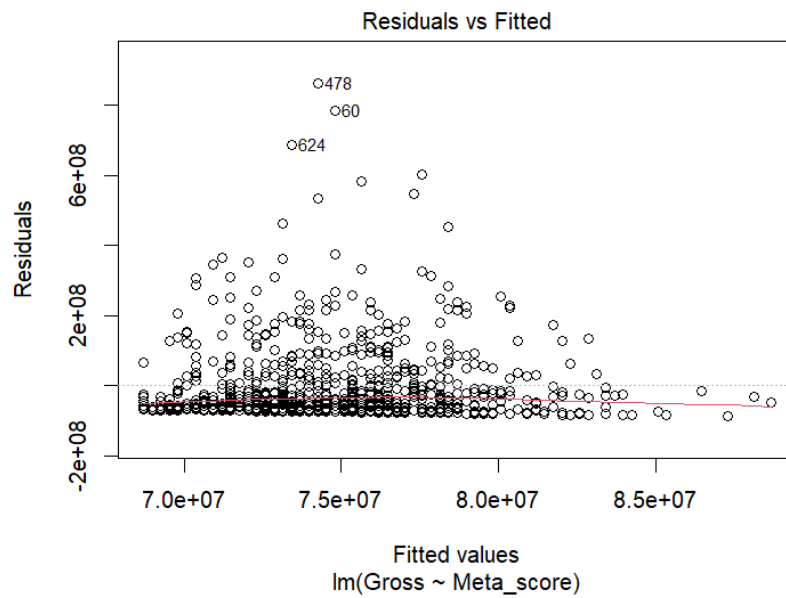
	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	96442843	26009397	3.708	0.000224 ***
Meta_score	-277444	331500	-0.837	0.402897

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 113400000 on 748 degrees of freedom
(250 observations deleted due to missingness)
Multiple R-squared: 0.0009356, Adjusted R-squared: -0.0004001
F-statistic: 0.7005 on 1 and 748 DF, p-value: 0.4029

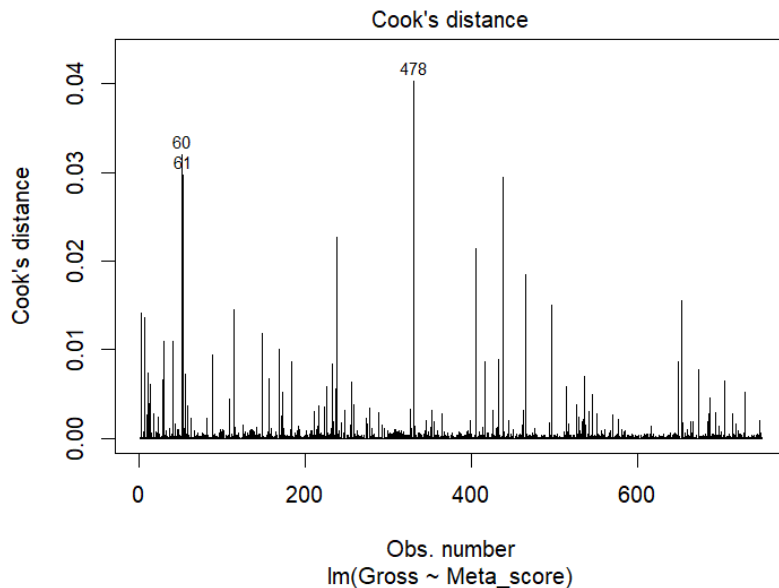
▪ ASSUMPTION CHECKS

```
plot(model_h2, which = 1)
plot(model_h2, which = 2)
shapiro.test(residuals(model_h2))
plot(model_h2, which = 4)
```



Shapiro-Wilk normality test

data: residuals(model_h2)
W = 0.66929, p-value < 2.2e-16



LOG TRANSFORM

```
> summary(model_h2_log)
```

Call:

```
lm(formula = log(Gross) ~ Meta_score, data = imdb_clean)
```

Residuals:

Min	1Q	Median	3Q	Max
-9.7705	-1.2894	0.5453	1.6425	3.9540

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	17.367776	0.515767	33.674	<2e-16 ***
Meta_score	-0.008299	0.006574	-1.263	0.207

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.248 on 748 degrees of freedom
(250 observations deleted due to missingness)

Multiple R-squared: 0.002126, Adjusted R-squared: 0.0007924

F-statistic: 1.594 on 1 and 748 DF, p-value: 0.2072

```
> shapiro.test(residuals(model_h2_log))
```

Shapiro-Wilk normality test

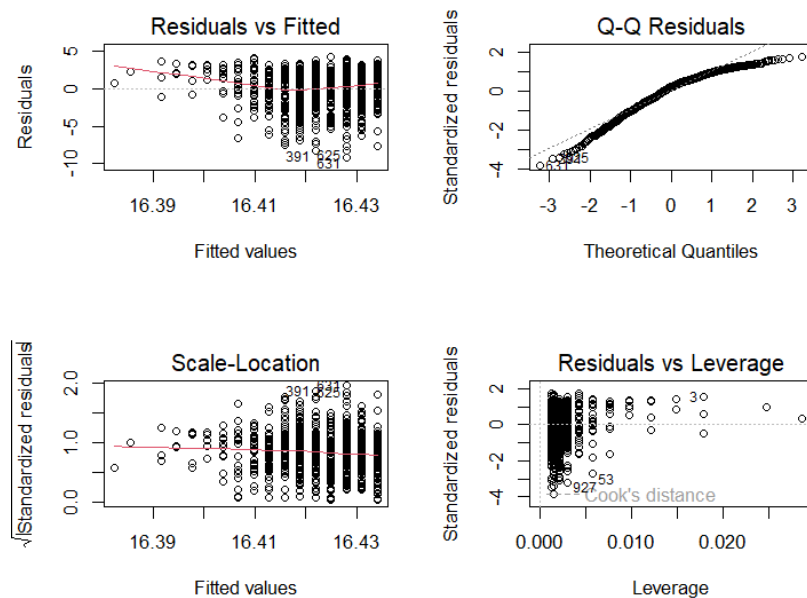
data: residuals(model_h2_log)

W = 0.93385, p-value < 2.2e-16

```
> par(mfrow = c(2, 2))
```

```
> plot(model_h2_log)
```

```
> par(mfrow = c(1, 1))
```



• H3 APPLICATION TEST:

```
> summary(model_h3)
              Df Sum Sq Mean Sq F value Pr(>F)
Primary_Genre  13   1.65  0.12724   1.692  0.0574 .
Residuals     986  74.17  0.07522
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> shapiro.test(residuals(model_h3))

      Shapiro-Wilk normality test

data:  residuals(model_h3)
W = 0.94287, p-value < 2.2e-16

> leveneTest(IMDB_Rating ~ Primary_Genre, data = imdb_clean)
Levene's Test for Homogeneity of Variance (center = median)
      Df F value Pr(>F)
group  13   1.946  0.02227 *
      986
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

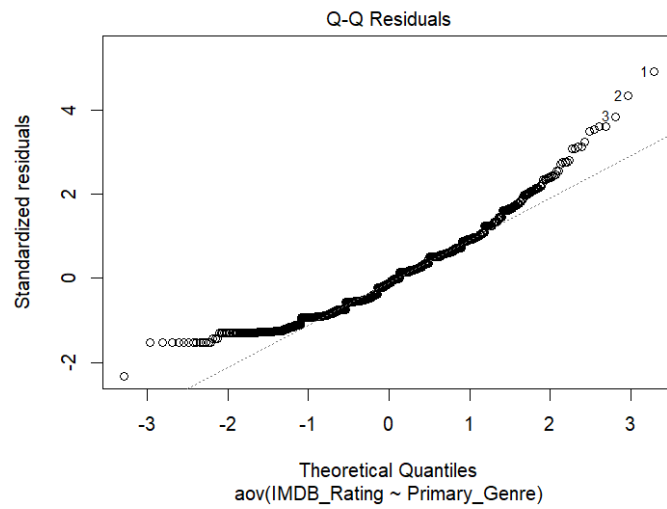
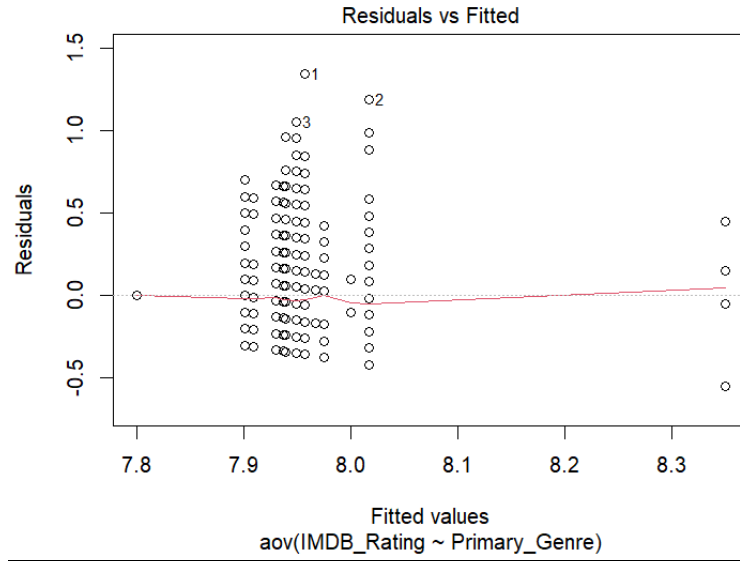
Warning message:
In leveneTest.default(y = y, group = group, ...) : group coerced to factor.

> kruskal.test(IMDB_Rating ~ Primary_Genre, data = imdb_clean)

      Kruskal-Wallis rank sum test

data:  IMDB_Rating by Primary_Genre
Kruskal-Wallis chi-squared = 14.567, df = 13, p-value = 0.3351
```

```
plot(model_h3, which = 1)
plot(model_h3, which = 2)
```



- H4 APPLICATION TEST:

```

> model_h4 <- lm(IMDB_Rating ~ Runtime, data = imdb_clean)
> summary(model_h4)

Call:
lm(formula = IMDB_Rating ~ Runtime, data = imdb_clean)

Residuals:
    Min       1Q   Median       3Q      Max
-0.53550 -0.20486 -0.03168  0.15282  1.30515

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  7.6563477   0.0379558  201.718  < 2e-16 ***
Runtime      0.0023838   0.0003011   7.917 6.44e-15 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2674 on 998 degrees of freedom
Multiple R-squared:  0.0591,    Adjusted R-squared:  0.05815
F-statistic: 62.68 on 1 and 998 DF,  p-value: 6.442e-15

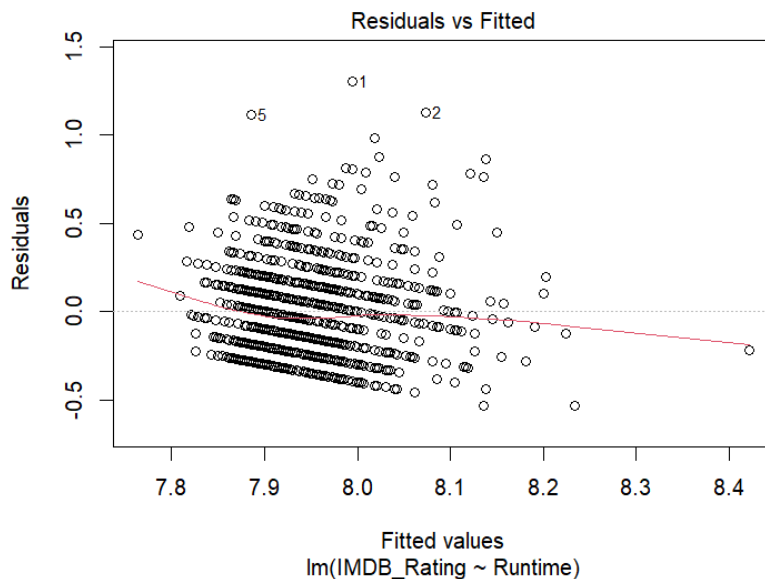
```

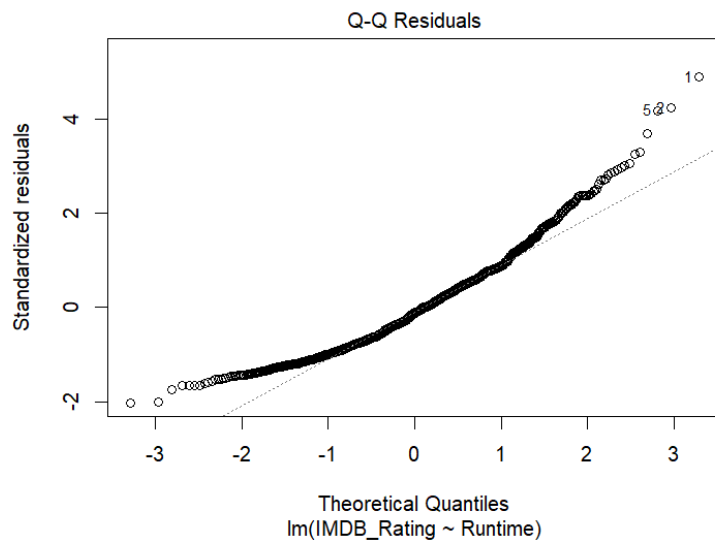
■ ASSUMPTION CHECKS

```

plot(model_h4, which = 1)
plot(model_h4, which = 2)
shapiro.test(residuals(model_h4))
plot(model_h4, which = 4)

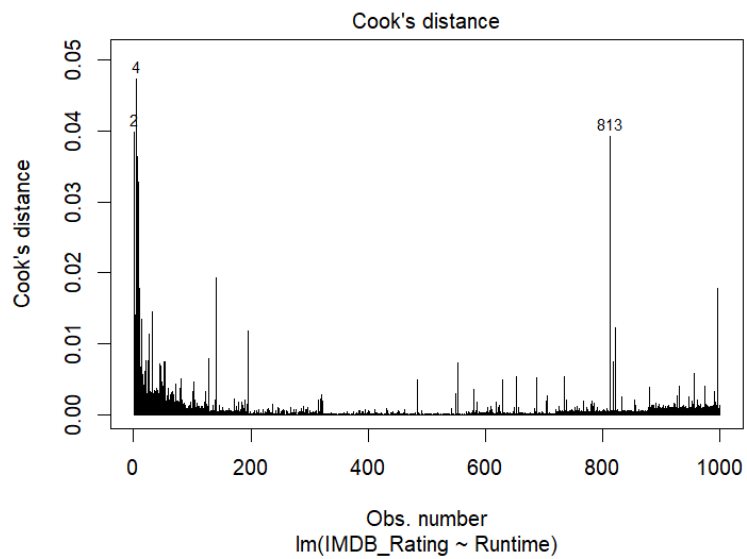
```





Shapiro-Wilk normality test

data: residuals(model_h4)
W = 0.95098, p-value < 2.2e-16



-
- H5 APPLICATION TEST:

```
> summary(model_h5)
      Df Sum Sq Mean Sq F value Pr(>F)
Certificate_Group 2 6.247e+17 3.124e+17 27.59 2.5e-12 ***
Residuals      828 9.373e+18 1.132e+16
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
169 observations deleted due to missingness
> shapiro.test(residuals(model_h5))
```

Shapiro-Wilk normality test

```
data: residuals(model_h5)
W = 0.72925, p-value < 2.2e-16
```

```
> leveneTest(Gross ~ Certificate_Group, data = imdb_clean)
Levene's Test for Homogeneity of Variance (center = median)
      Df F value    Pr(>F)
group  2  30.445 1.743e-13 ***
      828
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Warning message:

In leveneTest.default(y = y, group = group, ...) : group coerced to factor.

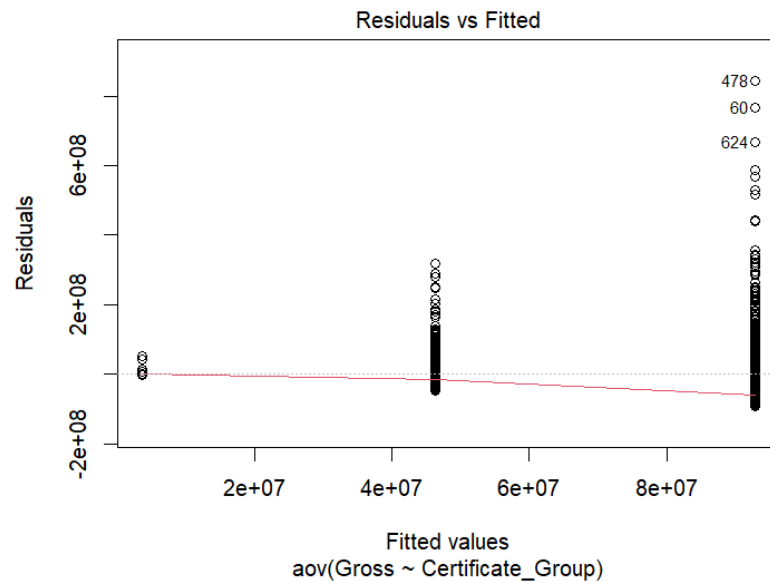
```
> kruskal.test(Gross ~ Certificate_Group, data = imdb_clean)
```

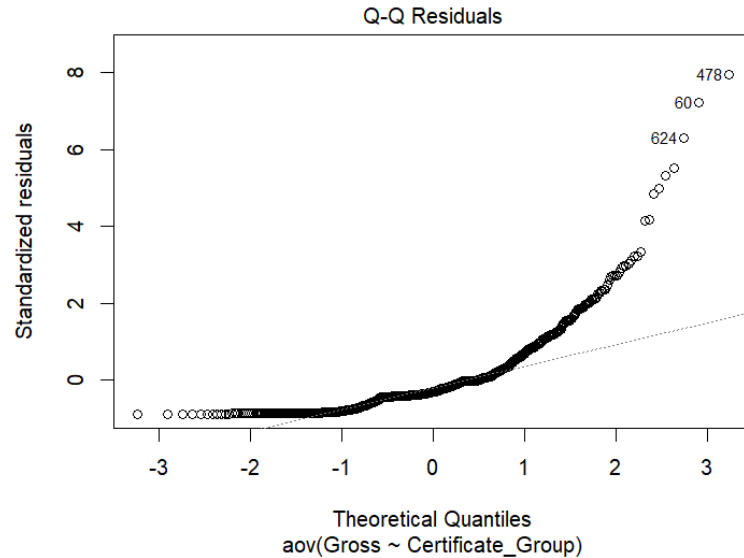
Kruskal-wallis rank sum test

```
data: Gross by Certificate_Group
Kruskal-wallis chi-squared = 86.531, df = 2, p-value < 2.2e-16
```

```
plot(model_h5, which = 1)
```

```
plot(model_h5, which = 2)
```





- **H6 APPLICATION TEST:**

```
> model_h6 <- lm(Gross ~ No_of_Votes, data = imdb_clean)
> summary(model_h6)
```

Call:
lm(formula = Gross ~ No_of_Votes, data = imdb_clean)

Residuals:

	Min	1Q	Median	3Q	Max
	-411693049	-31474027	-17057425	11272557	768545088

Coefficients:

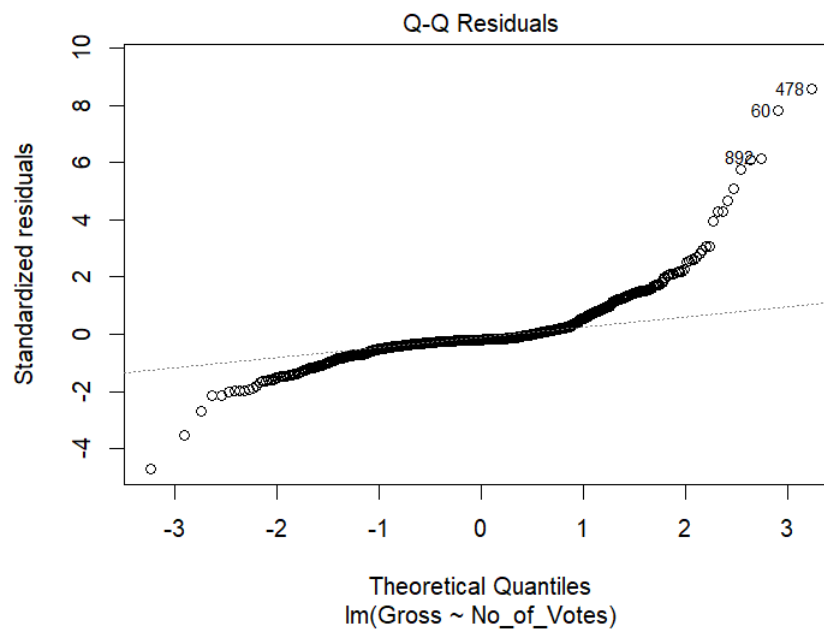
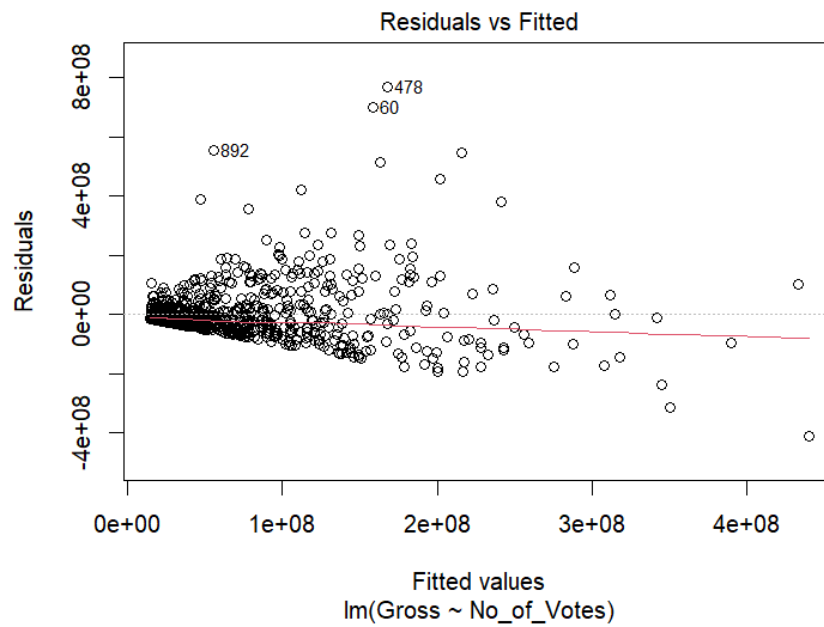
	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.020e+07	4.233e+06	2.411	0.0161 *
No_of_Votes	1.834e+02	9.080e+00	20.203	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 89890000 on 829 degrees of freedom
(169 observations deleted due to missingness)
Multiple R-squared: 0.3299, Adjusted R-squared: 0.3291
F-statistic: 408.2 on 1 and 829 DF, p-value: < 2.2e-16

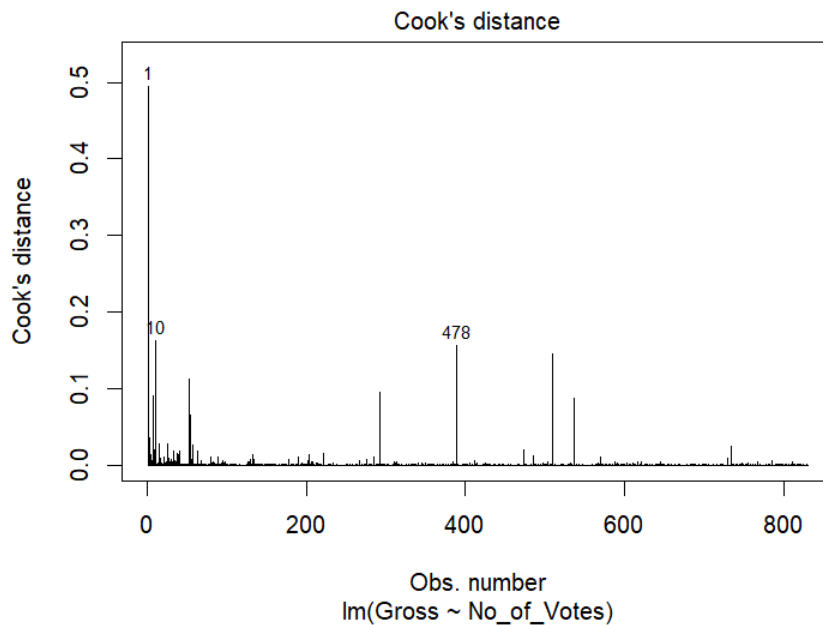
- **ASSUMPTION CHECKS**

```
plot(model_h6, which = 1)
plot(model_h6, which = 2)
shapiro.test(residuals(model_h6))
plot(model_h6, which = 4)
```



Shapiro-wilk normality test

```
data: residuals(model_h6)
W = 0.74354, p-value < 2.2e-16
```



LOG TRANSFORMATION

```
> summary(model_h6_log)
```

Call:

```
lm(formula = log(Gross) ~ IMDB_Rating, data = imdb_clean)
```

Residuals:

Min	1Q	Median	3Q	Max
-9.2541	-1.4279	0.5549	1.7819	4.2328

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	16.66501	2.35064	7.090	2.88e-12 ***
IMDB_Rating	-0.03038	0.29560	-0.103	0.918

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.412 on 829 degrees of freedom

(169 observations deleted due to missingness)

Multiple R-squared: 1.274e-05, Adjusted R-squared: -0.001194

F-statistic: 0.01056 on 1 and 829 DF, p-value: 0.9182

```
> shapiro.test(residuals(model_h6_log))
```

Shapiro-Wilk normality test

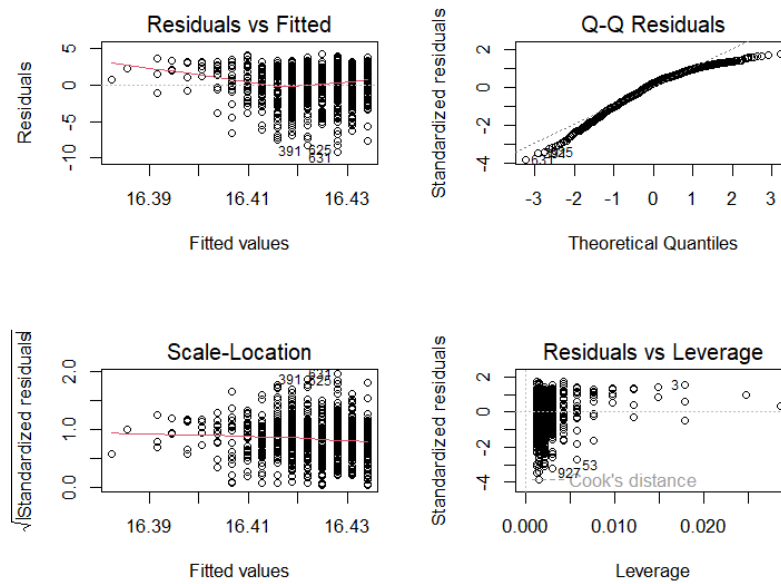
data: residuals(model_h6_log)

W = 0.94431, p-value < 2.2e-16

```
par(mfrow = c(2, 2))
```

```
plot(model_h6_log)
```

```
par(mfrow = c(1, 1))
```



• H7 APPLICATION TEST:

```
> summary(model_h7)
      Df Sum Sq Mean Sq F value    Pr(>F)
Decade   10   2.27  0.22710     3.054 0.000795 ***
Residuals 989  73.55  0.07437
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> shapiro.test(residuals(model_h7))

      Shapiro-Wilk normality test

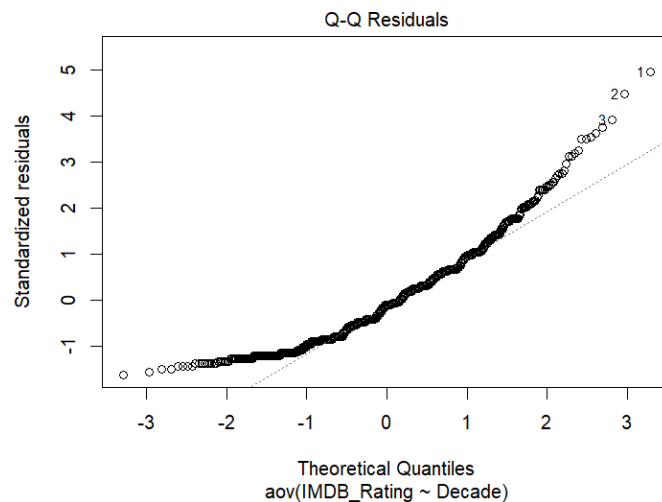
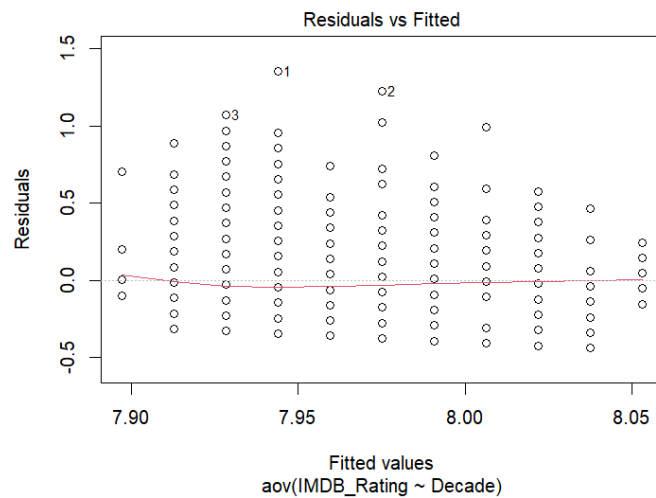
data:  residuals(model_h7)
W = 0.93308, p-value < 2.2e-16

> leveneTest(IMDB_Rating ~ Decade, data = imdb_clean)
Levene's Test for Homogeneity of Variance (center = median)
      Df F value    Pr(>F)
group  10  2.1498 0.01876 *
      989
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> kruskal.test(IMDB_Rating ~ Decade, data = imdb_clean)

      Kruskal-Wallis rank sum test

data:  IMDB_Rating by Decade
Kruskal-Wallis chi-squared = 39.948, df = 10, p-value = 1.731e-05

plot(model_h7, which = 1)
plot(model_h7, which = 2)
```



7. MODEL OUTPUT INTERPRETATIONS, INCLUDING PERFORMANCE AND CONFIDENCE

- H1 OUTPUT:

For our relationship we can see a p-value of $0.00565 < 0.05$ which shows significant positive relationship between variables. Our R-Squared is of 0.009201, but this explains less than 1% of the variance of our Gross. The assumption checks revealed a violation on the normality and signs of heteroscedasticity in the residual vs fitted plot.

As a response to these violations, we log-transformed the Gross variable and the model re-fit. Now with the log-transformed models, we can see that the p-values $0.918 > 0.05$ which tells us that the relationship is no longer statistically significant and with the new R-Squared of 0.00001

there is virtually no explanation of our Gross, much like previously. The log-transformation noticeably improved the homoscedasticity of residuals, as shown by the new residual vs. fitted plots and scale-location plots. It also improved the normality considerably.

We fail to reject H_0 due to the lack of statistical evidence of a relationship between the IMDB_Rating and Gross revenue after the log-transformation.

- H2 OUTPUT:

The relationship has a p-value of $0.402 > 0.05$ indicating a statically insignificant relationship. The R-Squared of 0.0009356 which like the previous one it explains virtually none of the variance in Gross. Assumption checks reveal violations on the normality signs of heteroscedasticity in the residuals vs. fitted plot.

Like H1, we log-transformed Gross and re-fit the model. In the log-transformation the p-value of $0.207 > 0.05$ remained insignificant and the R-Squared remained near zero at 0.0021426. The normality was improved with the log-transformation the residuals still deviate from normality.

Unlike H1, we reject H_0 before and after the log-transformation confirming there is no statistical evidence of a relationship between the Meta_Score and Gross revenue

- H3 OUTPUT:

The simple one-way ANOVA test, shows p-value of $0.0574 > 0.05$ meaning a marginally non-significant relationship. However, the assumption checks revealed violations of normality and homogeneity of variance, the latter was most driven by the imbalance noticed in EDA.

Due to this violation a Kruskal-Wallis test was run as a non-parametric alternative that does not assume equal variances. This test found no significant difference in IMDB_Rating across genres, this provided a clearer result than the ANOVA on its own.

We fail to reject H_0 , since there is no evidence of statistical significance between IMDB_Rating across genres.

- H4 OUTPUT:

Our p-value of $6.44e-15 < 0.05$ shows a significant positive relationship between our variables. The R-Squared of 0.0591 is modest but it explains 5.9% of the variance in IMDB_Rating. Assumption checks showed the

Shapiro-Wilk test was still significant, but due to the high W value = 0.951 and the small deviation in the upper tail of the Q-Q plot there is only a slight departure of normality. The residual vs fitted plot showed no big heteroscedasticity concerns. Given the small violations on the assumptions, a log-transformation wasn't needed.

Overall, we reject H_0 as there is statistically significant evidence of a relationship between IMDB_Rating and Runtime.

- H5 OUTPUT:

ANOVA found a significant relationship with a p-value of $2.5e-12 < 0.05$, indicating that Gross revenues does differ greatly across certificates categories. Assumption checks showed violations of normality and homogeneity of variance, this is consistent with strong right-skew of Gross and the small size of the "Not Rated" category.

As a response to these violations, we ran a Kruskal-Wallis test was run as a non-parametric alternative. The test helped confirm a stronger significant relationship between variables. Unlike H_3 , both parametric and non-parametric tests agree that the group differences are genuine.

We reject H_0 , because we have evidence of visibly statistical evidence that Gross revenue varies significantly across certificates. This result should still be seen with caution due to the sample size of "Not Rated"

- H6 OUTPUT:

For H_6 , our p-value $2.2e-16 < 0.05$ indicates a strong positive relationship, with No_of_Votes explaining 33% of the variance of Gross (R-Squared of 0.3299) by far the strongest relationship found in all the linear regression tests, consistent with our 0.57 correlation. However, there are clear violations of normality in the assumption checks and heteroscedasticity in the residuals vs. fitted plot.

Gross was log-transformed and the model re-fit. The log-transformation confirmed that the relationship remained strongly significant, with just a slight reduce R-Squared. Normality improved considerably, though residuals still show from deviation from normality.

We reject H_0 due to the strong significant relationship that remained before and after log-transformation between No_of_Votes and Gross.

- H7 OUTPUT:

We find a statistically significant result with a p-value of $0.000795 < 0.05$ indicating that IMDB_Rating varies greatly across release decades. Like previous tests, assumption checks revealed violations of normality and homogeneity of variance, consistent with the uneven distribution of films across decades.

Again, a Kruskal-Wallis test was run as a non-parametric alternative. The test confirmed the significant difference between variables. Like H5, both tests agree this suggests the decade effect is genuine.

We reject H_0 , as there is visibly statistical evidence that mean IMDB_Rating has changed across different release decades. Results from decades with fewer films should be interpreted with caution

GITHUB REPO - https://github.com/lorenav1901/applied_statistical_modeling

DATASET LINK - <https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows>